

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**



ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И УПРАВЛЯЮЩИХ СИСТЕМ

Лабораторная работа №2
по дисциплине: **Операционные системы**
тема: **«Виртуальная память в ОС Linux»**

Выполнил: ст. группы ВТ-231
Масленников Д. А.

Проверили:
Островский А. М.

Белгород 2025 г

Лабораторная работа № 2

Цель работы: Изучить концепцию виртуальной памяти в Linux, механизмы отображения виртуальных адресов в физические, организацию памяти процесса, работу подкачки и кэширования.

Цель работы обуславливает постановку и решение следующих задач:

1. Изучить концепцию виртуальной памяти в ОС Linux: назначение, преимущества, отличие от физической памяти.
2. Ознакомиться с механизмами отображения виртуальных адресов в физические.
3. Рассмотреть организацию адресного пространства процесса: сегменты кода, данных, кучи, стека, область динамически подгружаемых библиотек (shared libraries).
4. Освоить использование системных вызовов для управления памятью:
 - 4.1. `mmap()`, `munmap()` — отображение файлов и анонимное отображение в память.
 - 4.2. `mlock()`, `munlock()`, `mlockall()`, `munlockall()` — блокировка страниц в памяти, предотвращение выгрузки в swar.
 - 4.3. `mprotect()` — изменение прав доступа к страницам (чтение/запись/исполнение).
 - 4.4. `madvise()` — передача советов ядру по предполагаемым шаблонам доступа (например, `MADV_SEQUENTIAL`, `MADV_RANDOM`, `MADV_DONTNEED`). `posix_madvise()` — аналогичный механизм в POSIX.
5. Изучить организацию взаимодействия функций стандартной библиотеки для динамического выделения памяти (`malloc()`, `calloc()`, `realloc()`, `free()`) с системными вызовами (`brk()`, `mmap()`).
6. Ознакомиться с механизмами работы ядра с памятью:
 - 6.1. Копирование-при-записи (Copy-on-Write, COW) — экономия памяти при порождении процессов через `fork()`, создание копий страниц только при изменении.
 - 6.2. Механизм подкачки страниц — перенос страниц из ОЗУ во вторичное хранилище (swar) и обратно; причины и последствия подкачки.
 - 6.3. Кэш страниц — роль в ускорении операций чтения / записи файлов, стратегии замещения страниц.
 - 6.4. Обработка отказов по странице (page fault): причины (отсутствие страницы в памяти, защита доступа) и реакции ядра.
7. Освоить консольные инструменты для анализа состояния памяти.
 - 7.1. `free` — просмотр использования оперативной памяти и swar-раздела.
 - 7.2. `vmstat` — мониторинг состояния памяти, подкачки и нагрузки на систему.
 - 7.3. `top` / `htop` — контроль за использованием памяти процессами.
 - 7.4. `/proc/[PID]/maps` и `/proc/[PID]/smaps` — исследование структуры адресного пространства процесса, распределение анонимной и файловой памяти.
 - 7.5. `/proc/meminfo` — сводная информация об использовании памяти в системе.
 - 7.6. `ptop` — наглядное отображение карт памяти для процесса.
8. Выполнить индивидуальное задание для закрепления знаний на практике, подготовив соответствующую программу на языке C (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % — остаток от деления; если остаток равен 0, студенту назначается задание с максимальным номером).
9. Подготовить отчёт по работе, который должен включать: краткое описание всех использованных системных вызовов и команд; текст программы; протоколы; скриншоты с демонстрацией поведения ОС Linux; выводы.

Вариант 2

Задание «Общий разум». Убедиться, что включен механизм KSM, предназначенный для снижения потребления памяти:

```
cat /sys/kernel/mm/ksm/run
```

Если вывод 0 — включить:

```
echo 1 | sudo tee /sys/kernel/mm/ksm/run
```

Проверить текущие показатели:

```
grep . /sys/kernel/mm/ksm/pages_*
```

Важно зафиксировать до эксперимента:

— pages_shared — количество объединённых страниц,

— pages_sharing — количество процессов, делящих эти страницы,

— full_scans — число полных проходов сканера KSM.

Убедиться, что KSM активен:

```
systemctl status ksmtuned
```

Для реализации задания необходимо написать программу, которая создает два процесса, которые

выделяют одинаковые блоки памяти, заполняют их идентичными данными и удерживают некоторое время.

Сравнить статистику /sys/kernel/mm/ksm/pages_* до и после.

Через несколько секунд после заполнения буфера ядро начнёт объединять страницы.

Показатели

pages_shared и pages_sharing должны увеличиться.

Выполнить замер для показателей: pages_shared, pages_sharing, full_scans, VmRSS (parent), VmRSS (child), CPU (ksmd)

Выполнение индивидуального задания:

Исходный код программы:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <fcntl.h>
#include <time.h>

// Размер буфера в байтах (например, 128 МБ, чтобы было заметно, но не слишком много)
// Подберите под свою систему, чтобы не вызвать OOM
#define BUFFER_SIZE (128 * 1024 * 1024)

// Функция для чтения значения из /sys/kernel/mm/ksm/*
long read_ksm_stat(const char *stat_name) {
    char path[256];
    snprintf(path, sizeof(path), "/sys/kernel/mm/ksm/%s", stat_name);
    FILE *fp = fopen(path, "r");
    if (!fp) {
        perror("fopen ksm stat");
        return -1;
    }
    long value;
```

```

    if (fscanf(fp, "%ld", &value) != 1) {
        value = -1;
    }
    fclose(fp);
    return value;
}

// Функция для вывода VmRSS процесса (в КБ)
long get_vm_rss(pid_t pid) {
    char path[256];
    snprintf(path, sizeof(path), "/proc/%d/status", pid);
    FILE *fp = fopen(path, "r");
    if (!fp) {
        perror("fopen proc status");
        return -1;
    }
    char line[256];
    long rss = -1;
    while (fgets(line, sizeof(line), fp)) {
        if (sscanf(line, "VmRSS: %ld kB", &rss) == 1) {
            break;
        }
    }
    fclose(fp);
    return rss;
}

// Функция для вывода текущих KSM статистик и VmRSS
void print_stats(pid_t parent_pid, pid_t child_pid) {
    printf("KSM Stats:\n");
    printf("  pages_shared: %ld\n", read_ksm_stat("pages_shared"));
    printf("  pages_sharing: %ld\n", read_ksm_stat("pages_sharing"));
    printf("  full_scans: %ld\n", read_ksm_stat("full_scans"));

    // CPU usage ksmd - это сложнее, используем top или ps вручную, здесь
    // пропустим
    // Для простоты можно добавить: ps -p $(pgrep ksmd) -o %cpu
    // Но для автоматизации вызовем system
    system("ps -p $(pgrep ksmd) -o %cpu | tail -n 1");

    printf("VmRSS (parent %d): %ld kB\n", parent_pid, get_vm_rss(parent_pid));
    if (child_pid > 0) {
        printf("VmRSS (child %d): %ld kB\n", child_pid, get_vm_rss(child_pid));
    }
}

int main() {
    // Убедимся, что KSM включен (должен быть выполнен echo 1 >
    // /sys/kernel/mm/ksm/run заранее)
    if (read_ksm_stat("run") != 1) {

```

```

        fprintf(stderr, "KSM не включен! Выполните: echo 1 | sudo tee
/sys/kernel/mm/ksm/run\n");
        return 1;
    }

    pid_t parent_pid = getpid();
    printf("До эксперимента:\n");
    print_stats(parent_pid, 0);

    // Fork для создания двух процессов
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }

    // В обоих процессах выделяем память
    void *buffer;
    if (pid == 0) { // Child
        printf("Child (%d): Выделяю и заполняю буфер...\n", getpid());
    } else { // Parent
        printf("Parent (%d): Выделяю и заполняю буфер...\n", getpid());
    }

    // Выделяем анонимную память с mmap (лучше для KSM, чем malloc)
    buffer = mmap(NULL, BUFFER_SIZE, PROT_READ | PROT_WRITE, MAP_PRIVATE |
MAP_ANONYMOUS, -1, 0);
    if (buffer == MAP_FAILED) {
        perror("mmap");
        return 1;
    }

    // Помечаем область как подходящую для KSM (MADV_MERGEABLE)
    if (madvise(buffer, BUFFER_SIZE, MADV_MERGEABLE) < 0) {
        perror("madvise MADV_MERGEABLE");
        munmap(buffer, BUFFER_SIZE);
        return 1;
    }

    // Заполняем буфер идентичными данными (простой паттерн для демонстрации)
    // Чтобы страницы были идентичны, заполняем одним и тем же во всех процессах
    srand(42); // Фиксированный seed для идентичности
    for (size_t i = 0; i < BUFFER_SIZE / sizeof(int); i++) {
        ((int *)buffer)[i] = rand(); // Идентичные случайные числа
    }

    // Чтобы страницы стали резидентными, коснемся их (touch)
    for (size_t i = 0; i < BUFFER_SIZE; i += getpagesize()) {
        ((volatile char *)buffer)[i] = ((char *)buffer)[i]; // Чтение для
резиденции
    }

```

```

if (pid > 0) { // Parent ждет немного, чтобы child тоже заполнил
    sleep(5);
}

// Держим паузу, чтобы ksmд успел просканировать и слить страницы
// KSM сканирует периодически, подождем 30-60 секунд
printf("Процесс %d: Держу буфер 60 секунд для KSM...\n", getpid());
sleep(60);

// Выводим статистику после
if (pid == 0) { // Child выводит свою
    printf("После в child:\n");
    print_stats(parent_pid, getpid());
} else { // Parent ждет child и выводит
    wait(NULL);
    printf("После в parent:\n");
    print_stats(parent_pid, 0); // Child уже завершился
}

// Освобождаем память
munmap(buffer, BUFFER_SIZE);
return 0;
}

```

Результат выполнения программы:

```

~/projects/university/5sem/os/lab2 master* > ./lab2
До эксперимента:
KSM Stats:
  pages_shared: 0
  pages_sharing: 0
  full_scans: 2
0.0
VmRSS (parent 8426): 1724 kB
Parent (8426): Выделяю и заполняю буфер...
Child (8431): Выделяю и заполняю буфер...
Процесс 8431: Держу буфер 60 секунд для KSM...
Процесс 8426: Держу буфер 60 секунд для KSM...
После в child:
KSM Stats:
  pages_shared: 32768
  pages_sharing: 32768
  full_scans: 5
0.0
VmRSS (parent 8426): 132796 kB
VmRSS (child 8431): 132568 kB
После в parent:
KSM Stats:
  pages_shared: 32768
  pages_sharing: 32768
  full_scans: 5
0.0
VmRSS (parent 8426): 132796 kB

```

Вывод:

В ходе выполнения задания №7 («Общий разум») было продемонстрировано, что механизм KSM (Kernel Samepage Merging) действительно объединяет идентичные страницы памяти из разных процессов, что подтверждается ростом счётчиков `pages_shared` и `pages_sharing` после запуска программы с одинаковыми данными в двух процессах. Это позволяет существенно снизить реальное потребление физической памяти (RSS), особенно при запуске множества похожих процессов, таких как виртуальные машины или контейнеры.